check_cm_go_toolchain_matches_builder

scripts/pre_build/check_cm_go_toolchain_matches_builder.sh — CM-GO-TOOLCHAIN-MATCHES-BUILDER static pre-build gate

Revision: 1 **Last modified:** 2026-08-21T00:00:00Z **Status:** active
Item: BOB-153

Purpose

Refuse any tree in which a Dockerfile's Go **builder image** cannot satisfy the `go` directive of the `go.mod` it builds.

The two values are a floor and its satisfier. They must move together — but nothing in this repository compared them, which is exactly why they diverged.

Forensic anchor (BOB-153)

`qBitTorrent-go/go.mod` declared `go 1.26.2` while `qBitTorrent-go/Dockerfile` built `FROM golang:1.23-alpine`, so the Go profile could not build at all:

```
go: go.mod requires go >= 1.26.2 (running go 1.23.12; GOTOOLCHAIN=local)
```

Both halves of that error are properties of the image, confirmed by reading the published image config from the registry (no pull needed):

image	GOLANG_VERSION	GOTOOLCHAIN
golang:1.23-alpine	1.23.12	local
golang:1.26-alpine	1.26.7	local
golang:1.26.2-alpine	1.26.2	local

The official Go images pin GOTOOLCHAIN=local deliberately, so an insufficient builder is a **hard failure** rather than a silent toolchain download.

git log -p --follow shows both values were introduced by the **same commit** (4002c57, 2026-04-22). This was never a drift over time — the two were born divergent and nothing ever compared them, so the defect sat unnoticed for four months until the profile was actually needed. That is the §11.4.227 prose-not-seam gap in its purest form: a real invariant nobody had written down as a check.

Beyond the build breakage, the mismatch blocked feature 002's quickstart Scenario 6, which exists to confirm the operator-owned-writes fix reaches every service (FR-016).

Why the directive was right and the builder was wrong

The go 1.26.2 directive is not an author's preference. It is forced by the dependency graph:

dependency	its own go directive
github.com/getkin/kin-openapi v0.136.0 (direct)	go 1.26
github.com/gin-gonic/gin v1.12.0 (direct)	go 1.25.0
modernc.org/sqlite v1.50.0 (direct)	go 1.25.0
golang.org/x/net v0.51.0	go 1.25.0

Go 1.21+ refuses to build a module whose dependency requires a newer toolchain, and go get **raises** the directive automatically to satisfy it. Reproduced in a throwaway module: go get github.com/getkin/kin-openapi@v0.136.0 wrote exactly go 1.26.2 unprompted, and forcing the directive back down to 1.23 under -mod=readonly made the build refuse to proceed.

So “lower the directive to match the builder” is not an available option; it is a dependency downgrade wearing a one-line disguise. The builder image was the value that was wrong.

What the gate asserts — the real condition, not a proxy

```
builder_toolchain_version  >=  go.mod `go` directive
```

Not string equality. Equality is the tempting implementation and it is wrong: it would refuse `golang:1.27-alpine` against a `go 1.26.2` directive even though that builder satisfies the floor completely. §11.4.201(1) makes such a false-**positive** refusal exactly as forbidden as a false pass — it halts real work and teaches people to route around gates. The comparison is therefore a real version comparison, in both directions.

The three-valued verdict

tag shape vs directive	verdict	why
newer major/minor, or equal minor with patch ≥	**ok**	satisfies the floor
older major/minor, or equal minor with patch <	**FAIL**	provably cannot build
tag names no patch, directive names patch 0	**ok**	nothing to prove
tag names no patch, directive names patch > 0	**WARN** (non-blocking)	floats; true today, not verifiable here
tag names no version (`latest`, `alpine`)	**FAIL**	unresolvable *and* unreproducible

The middle value exists for a reason. `golang:1.26-alpine` resolves today to 1.26.7, which satisfies a `go 1.26.2` floor — but **nothing in the tree proves that**; statically `1.26` reads as 1.26.0, which does not. Failing it would refuse a build that demonstrably works (§11.4.201(1)); passing it silently would assert something unverified (§11.4.6). So it is a non-blocking WARN naming the fix, following the precedent `check_cm_plugin_count.sh` set for the README badge and §11.4.234’s rule that a pre-build gate must not block a build over something that is not broken.

`qBitTorrent-go/Dockerfile.jackett` is exactly this shape today and is reported as a WARN, not a FAIL.

Carrier vs thing

Only a real instruction line matches — `^[[:space:]]*FROM[[:space:]]+golang:`. A comment *mentioning* an old builder ("this used to be `FROM golang:1.23-alpine`") is a carrier, not a builder, and must not trip the gate (§11.4.201(7)(a)). A substring scan for `golang:1.23` would fail that distinction; the paired meta-test's `good-carrier` fixture holds the gate to it.

Control needles

A quiet zero from a blind extractor is indistinguishable from a clean tree (§11.4.201(6)). Before any result is trusted, both extractors are proven able to **see**: a synthetic `FROM golang:` stage is injected into an in-memory copy of each Dockerfile and the extracted count must rise by exactly one, and a synthetic directive is injected into each `go.mod` and must be read back. Zero discovered pairs is likewise a FAIL.

> The needle earned its keep immediately. `qBitTorrent-go/Dockerfile` ends **without** a trailing newline (last byte `]`), so the first implementation's `echo`-based injection glued the synthetic stage onto the end of `CMD [...]` and the extractor correctly did not see it. The needle caught a flaw in its own injection before any verdict was trusted — which is precisely what a needle is for. The injection now leads with `\n`.

Scope

First-party modules only. `submodules/` and `constitution/` are consumed by reference and owned upstream (§11.4.28/§11.4.177); refusing this repository's build over an upstream repo's Dockerfile would be a refusal its owner here cannot fix. A Go module with no golang-builder Dockerfile (`cmd/boba-ctl`, built on the host) has no builder image to reconcile and is correctly not a pair.

Pairing walks **up** from each Dockerfile's directory to the nearest `go.mod`. That matches how the build context is declared — `docker-compose.yml` sets `context: ./qBitTorrent-go` for both of that directory's Dockerfiles — so the module found is the module the builder actually compiles.

Usage

```
scripts/pre_build/
        check_cm_go_toolchain_matches_builder.sh              #
        this repository
scripts/pre_build/check_cm_go_toolchain_matches_builder.sh --root
        DIR   # treat DIR as root
scripts/pre_build/check_cm_go_toolchain_matches_builder.sh
        -v           # show every pair
scripts/pre_build/check_cm_go_toolchain_matches_builder.sh --help
```

Inputs: optional `--root DIR`; no stdin; no env input. **Outputs:** per-pair verdict on stdout, the `PASS` line always last (the pre-build wiring reads it with `tail -n1`); findings and the `FAIL` summary on stderr. **Side-effects:** none. Read-only; writes only to `mktemp` files it removes. Signals nothing, so §11.4.263 has no surface here. **Dependencies:** bash, sed, grep, find, wc, mktemp.

Exit codes

code	meaning
0	PASS — ≥1 pair checked, every builder satisfies its directive
1	FAIL — a mismatch, an unversioned tag, or zero pairs discovered
2	ERROR — usage error, or a control needle proved an extractor blind

Edge cases

- **`find` here is `bfs`, not GNU findutils.** Only portable primaries (`-name`/`-type`/`-prune`/`-print`) are used; relative `-newermt` forms are rejected by bfs while printing nothing to stdout, which would turn a hard failure into a clean-looking empty result. `find`'s stderr is therefore treated as fatal rather than ignored.
- **Multi-stage Dockerfiles** with several `FROM golang:` stages are each checked; the count in the PASS line is builder *stages*, not files.
- **A golang builder with no enclosing `go.mod`** is reported only under `-v` and does not fail the build — that is not this gate's invariant.

Paired §1.1 mutation test

`tests/pre_build/test_check_cm_go_toolchain_matches_builder.sh` — 13 checks across hermetic fixture roots plus a real-tree smoke run:

- 5 golden-good (including the **false-positive guard**: a strictly newer builder must not be refused, and the **carrier** fixture)
- 1 warn fixture asserting both the exit code **and** that the WARN is actually emitted (a silently dropped warn is a §11.4.201(6) false-null)
- 5 golden-bad (including `bad-bob153`, the exact pre-fix shape, as a permanent regression guard, and `bad-inverted` for the likelier future recurrence where someone raises `go.mod` and forgets the Dockerfile)
- 1 blind-instrument guard (zero pairs must fail loudly)
- the real checkout

```
bash tests/pre_build/test_check_cm_go_toolchain_matches_builder.sh
```

Related

- `scripts/pre_build/check_cm_plugin_count.sh` — the non-blocking-WARN precedent
- `scripts/pre_build/check_cm_healthcheck_covers_served_ports.sh` — sibling pre-build gate
- `docs/workable_items.db` — BOB-153

Last verified: 2026-08-21 (gate GREEN on the real tree; meta-test 13/13).

> **Export status (§11.4.65):** the `.html`, `.pdf` and `.docx` twins of this guide were generated automatically by the repository's documentation export hook shortly after this file landed. They are derived artifacts — edit this `.md` and let the hook regenerate them; do not hand-edit the twins.